Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#6)

Use of Venus Simulator for RISC-V Procedure Execution

Student Name: **AHMED JAVED QURESHI** Roll No**: 24i-6554** Section: **CE-5A**

Date performed: **28 Sep, 2026**

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Verified by: Dr. Nasir Ali Shah Updated: Fall 2026

Task 1:

Description:

The original code had two problems. The procedure changed s0 (a saved register) to 3 and never put 2500 back, so the final addition used the wrong value and gave 1006 instead of 3503. Also, main had no exit, so after printing, the program ran into procedure again and messed up the registers.

In my fix, the procedure saves s0 on the stack before using it and restores it before returning, so main still sees 2500. I also added an exit call after the print. Now t1 = 2500 + 1003 = 3503 and the program stops cleanly.

Code:

```
.text

main:

   li   a0, 1

   li   s0, 2500

   jal  ra, procedure     # call procedure

   add  t1, s0, a0       # 2500 + 1003 = 3503



   li   a0, 1           # print int

   mv   a1, t1

   ecall



   li   a0, 10          # exit (stops it falling into procedure)
```

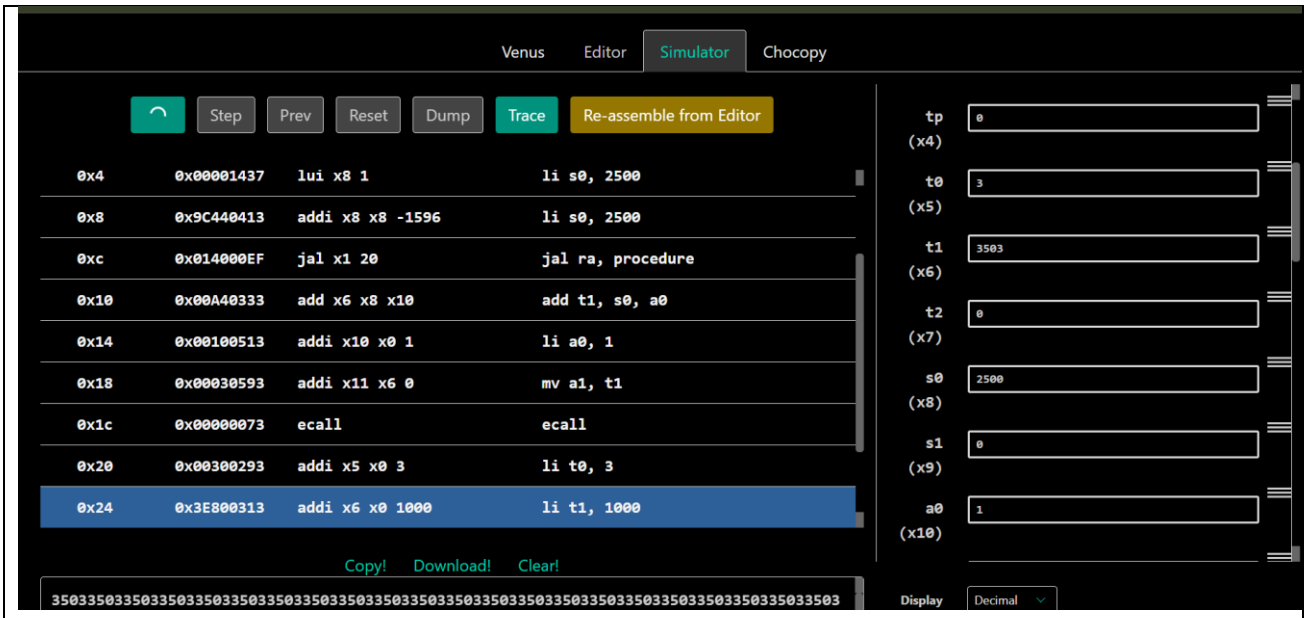
```
    ecall


procedure:
    addi sp, sp, -4       # make room on stack
    sw   s0, 0(sp)        # save s0 (callee-saved)

    li   s0, 3
    li   t0, 1000
    add  a0, s0, t0       # a0 = 1003

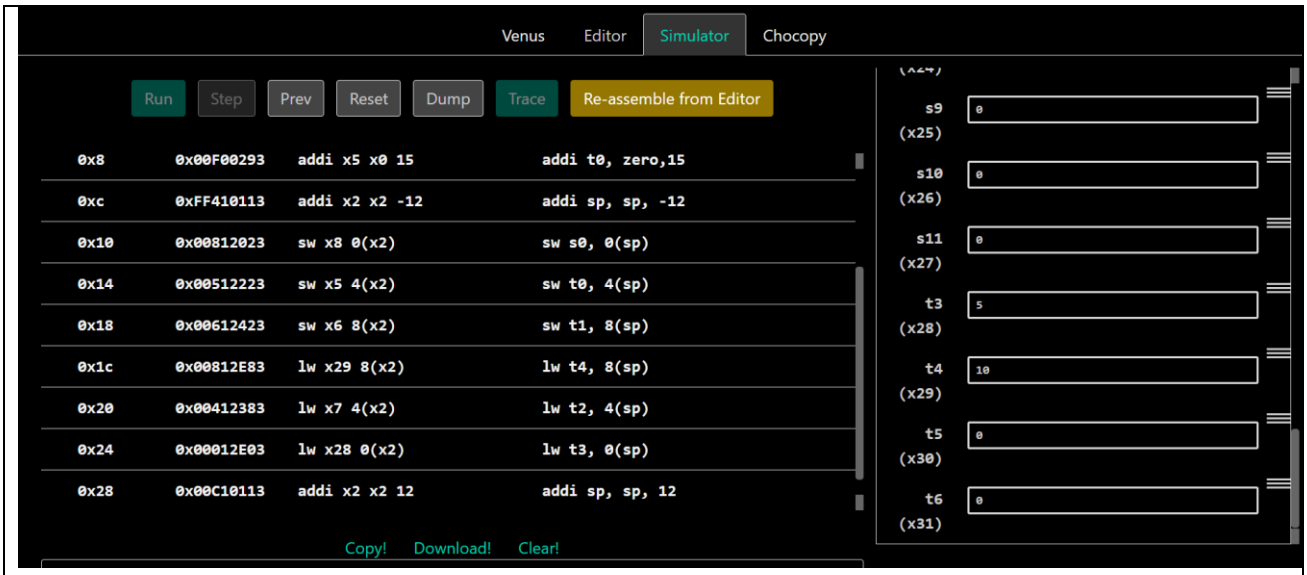
    lw   s0, 0(sp)        # restore s0 = 2500
    addi sp, sp, 4        # free stack
    jalr zero, ra, 0      # return
```

Output:


Venus
Editor
Simulator
Chocopy
Step
Prev
Reset
Dump
Trace
Re-assemble from Editor
0x4 0x00001437 lui x8 1 li s0, 2500
0x8 0x9C440413 addi x8 x8 -1596 li s0, 2500
0xc 0x014000EF jal x1 20 jal ra, procedure
0x10 0x00A40333 add x6 x8 x10 add t1, s0, a0
0x14 0x00100513 addi x10 x0 1 li a0, 1
0x18 0x00030593 addi x11 x6 0 mv a1, t1
0x1c 0x00000073 ecall ecall
0x20 0x00300293 addi x5 x0 3 li t0, 3
0x24 0x3E800313 addi x6 x0 1000 li t1, 1000
Copy!
Download!
Clear!
tp (x4)
t0 (x5)
t1 (x6)
t2 (x7)
s0 (x8)
s1 (x9)
a0 (x10)
Display
Decimal

Task 2:

Description:
1. The first three lines put 5, 10 and 15 into s0, t1 and t0. **2. addi sp, sp, -12 moves the stack pointer down by 12 bytes. The stack grows toward lower addresses, so this makes room for three 4-byte words.** **3. The three sw instructions copy the registers into that space, using offsets from sp. This is the push.** **4. The three lw instructions read the values back into t4, t2 and t3. This is the pop. I loaded them into different registers than the originals to show the data really came back from memory.** 5. **addi sp, sp, 12 moves sp back up, so the stack is empty again, the same as before the program started**.
Code:
addi s0, zero, 5 addi t1, zero, 10 addi t0, zero,15 addi sp, sp, -12 sw s0, 0(sp) sw t0, 4(sp) sw t1, 8(sp) lw t4, 8(sp) lw t2, 4(sp) lw t3, 0(sp) addi sp, sp, 12
Output:



Task 3:

Description:
The program calls EqSolver with a = 7 and b = 3, passing them in a0 and a1. The procedure computes (a+b) and (a-b) in two temporary registers, then multiplies them with mul. The result goes back in a0, so the caller copies it to s0 and prints it. Since 10 × 4 = 40, the console shows 40, and s0 holds 0x28.
Code:

```
.text

main:

   li   a0, 7          # a
```

```
    li   a1, 3          # b

    jal  ra, EqSolver

    mv   s0, a0         # x = result (40)



    li   a0, 1

    mv   a1, s0

    ecall

    li   a0, 10

    ecall


EqSolver:

    add  t0, a0, a1      # a + b

    sub  t1, a0, a1      # a - b

    mul  a0, t0, t1

    jalr zero, ra, 0
```

Output:



Task 4:

Description:
The side of the square (6) is kept in s0, which stays safe because neither procedure touches it. AreaFinder multiplies the side by itself, and PerimeterFinder shifts it left by 2 (the same as times 4). Both get the side in a0 and return their answer in a0. Main saves the area in s1 and the perimeter in s2, then prints them on separate lines. The output is 36 and 24.
Code:
.text

```
main:
   li   s0, 6          # side


   mv   a0, s0
   jal  ra, AreaFinder
   mv   s1, a0         # area = 36


   mv   a0, s0
   jal  ra, PerimeterFinder
   mv   s2, a0         # perimeter = 24

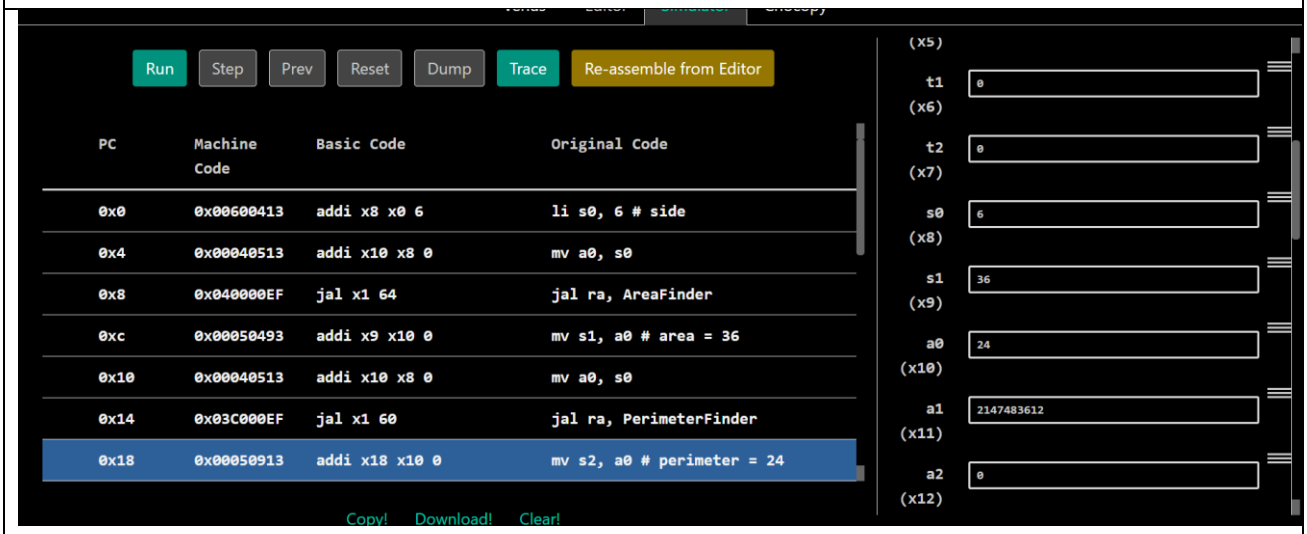

   li   a0, 1
   mv   a1, s1
   ecall               # print area
   li   a0, 11
   li   a1, 10
   ecall
   li   a0, 1
   mv   a1, s2
   ecall               # print perimeter
```

```
    li   a0, 10
    ecall

AreaFinder:
    mul  a0, a0, a0       # side * side
    jalr zero, ra, 0

PerimeterFinder:
    slli a0, a0, 2        # side * 4
    jalr zero, ra, 0
```

Output:



Task 5:

Description:
This evaluates x^3 + 5x^2 + 6x, which I rearranged into ((x+5)·x + 6)·x, so it takes only two multiplies. The procedure uses just one temporary register, so nothing needs to be saved on the stack. With x = 2, the result is 40, which matches 8 + 20 + 12. The value is passed in a0 and returned in a0
Code:

```
.text

main:

   li   a0, 2          # x

   jal  ra, EqSolver

   mv   s0, a0         # y = 40



   li   a0, 1

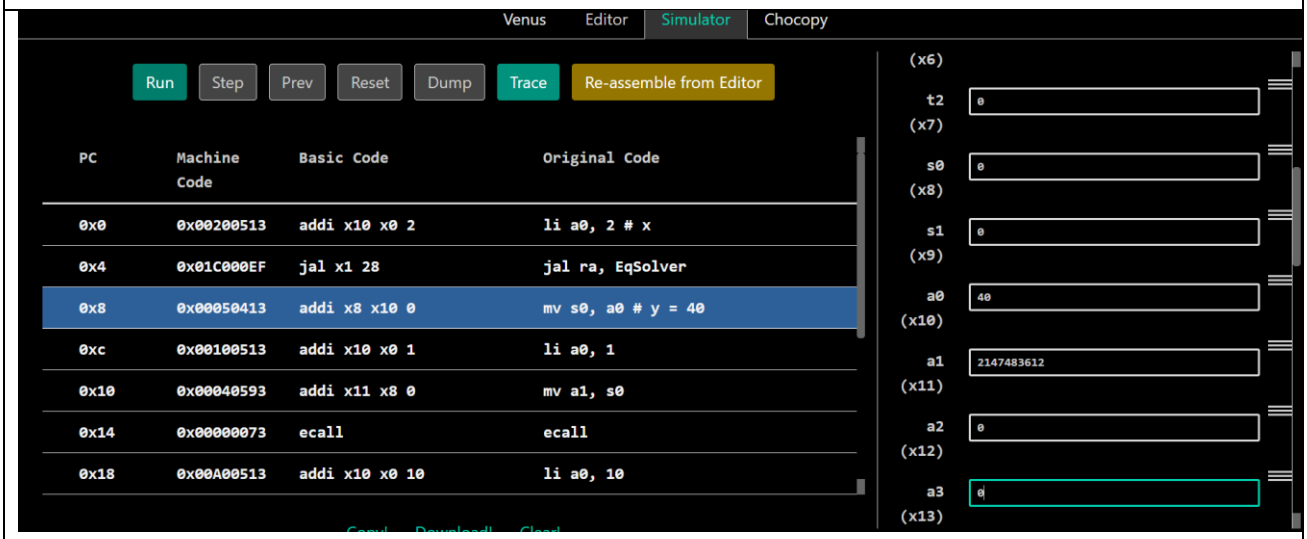
   mv   a1, s0

   ecall
```

```
    li   a0, 10
    ecall


EqSolver:
    addi t0, a0, 5        # x + 5
    mul  t0, t0, a0       # (x+5)*x
    addi t0, t0, 6        # (x+5)*x + 6
    mul  a0, t0, a0       # * x
    jalr zero, ra, 0
```

Output:



Task 6:

Description:
Main stores n in s0 and k in s1, then follows the if / else-if / else with branches. If k is 1 it calls F1 (x^2 + z^2), else if n > k it calls F2 (z^3), otherwise it calls F3 (w + 5). Each branch ends with a jump to done, so only one function runs. The result comes back in a0 and is printed. With n = 5 and k = 2 it picks F2 and prints 125.

Code:

```
.text

main:

   li   s0, 5          # n

   li   s1, 2          # k



   li   t0, 1

   bne  s1, t0, not_k1    # if (k == 1)

   mv   a0, s0

   mv   a1, s1

   jal  ra, F1
```

```
   j   done


not_k1:
   bge  s1, s0, else_part # else if (n > k)  (skip if k >= n)
   mv   a0, s0
   jal  ra, F2
   j   done


else_part:
   mv   a0, s0
   jal  ra, F3


done:
   mv   s2, a0          # y
   li   a0, 1
   mv   a1, s2
   ecall
   li   a0, 10
   ecall
```

```
F1:             # x^2 + z^2

   mul  t0, a0, a0

   mul  t1, a1, a1

   add  a0, t0, t1

   jalr zero, ra, 0


F2:             # z^3

   mul  t0, a0, a0

   mul  a0, t0, a0

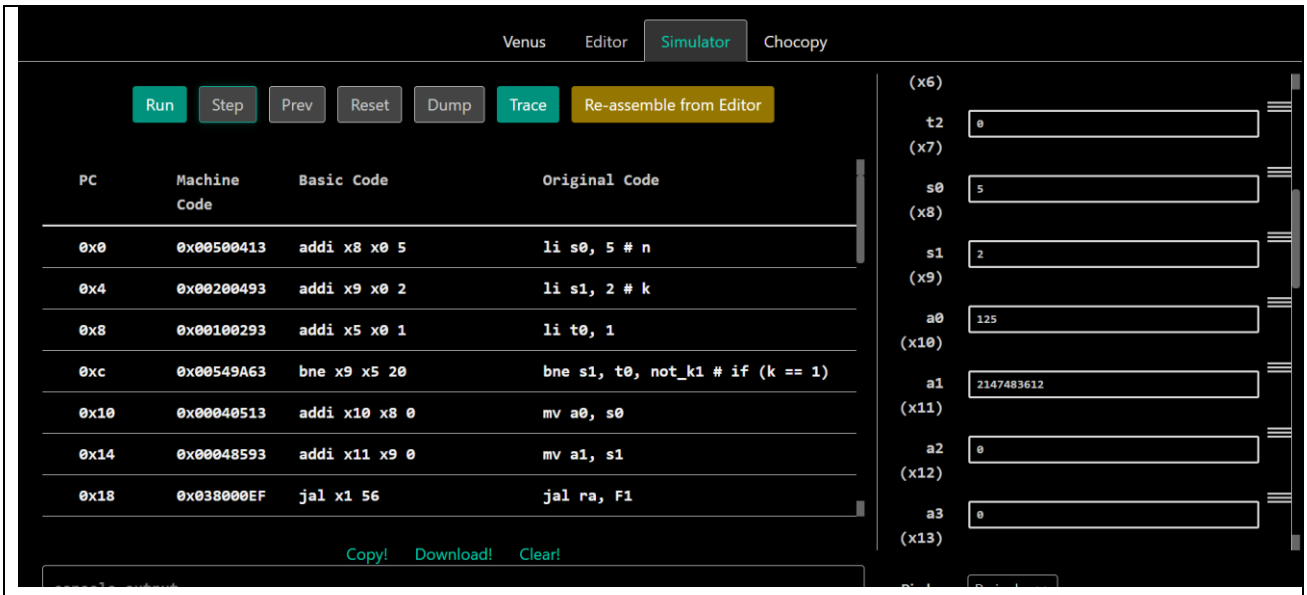
   jalr zero, ra, 0


F3:             # w + 5

   addi a0, a0, 5

   jalr zero, ra, 0
```

Output:



Task 7:

Description:
This is the same idea as Task 6 with different conditions. If k is 0 it calls fun1 (x^2), else if n equals k it calls fun2 (z^3), otherwise fun3 (w + 1). I used bne to skip over each case that doesn't match, and j done after each call. With n = 4 and k = 4 it takes the fun2 branch and prints 64.
Code:
.text main:

```
    li   s0, 4          # n
    li   s1, 4          # k

    bne  s1, zero, not_k0  # if (k == 0)
    mv   a0, s0
    jal  ra, fun1
    j    done

not_k0:
    bne  s0, s1, else_part # else if (n == k)
    mv   a0, s0
    jal  ra, fun2
    j    done

else_part:
    mv   a0, s0
    jal  ra, fun3

done:
    mv   s2, a0         # y
```

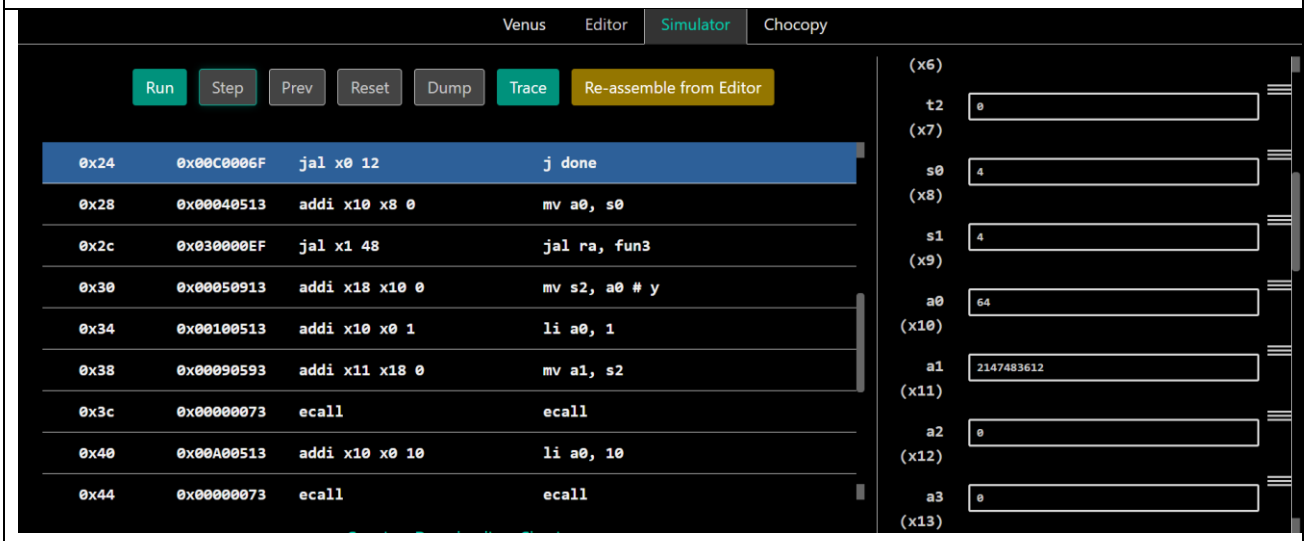
```
    li   a0, 1
    mv   a1, s2
    ecall
    li   a0, 10
    ecall


fun1:               # x^2
    mul  a0, a0, a0
    jalr zero, ra, 0


fun2:               # z^3
    mul  t0, a0, a0
    mul  a0, t0, a0
    jalr zero, ra, 0


fun3:               # w + 1
    addi a0, a0, 1
    jalr zero, ra, 0
```

Output:



Task 8:

Description:

(a) This is Fibonacci with a loop. a and b start at 0 and 1, and the loop runs from i = 2 up to n, each time doing c = a + b, a = b, b = c. If n is 0 it returns a directly. Otherwise it returns b when the loop ends. No stack is needed because there are no nested calls, and fib(7) gives 13.

(b) The recursive version calls itself for n-1 and n-2, then adds the two results. Because of this, each call has to save ra (its return address), s0 (its n) and s1 (the first result) on the stack, and restore them before returning. For n less than 2 it simply returns n, which covers both base cases at once. Each recursive call gets its own stack frame, and the frames unwind when the calls finish. fib(7) gives 13.

Code:

(a)

```
.text

main:

   li   a0, 7          # n

   jal  ra, fib

   mv   s0, a0         # y



   li   a0, 1

   mv   a1, s0

   ecall               # print y



   li   a0, 10

   ecall               # exit



fib:

   li   t0, 0          # a

   li   t1, 1          # b

   beq  a0, zero, ret_a   # n == 0

   li   t2, 2          # i

loop:

   blt  a0, t2, ret_b     # stop when i > n
```

```
   add  t3, t0, t1       # c = a + b

   mv   t0, t1          # a = b

   mv   t1, t3          # b = c

   addi t2, t2, 1       # i++

   j    loop

ret_a:

   mv   a0, t0

   jalr zero, ra, 0

ret_b:

   mv   a0, t1

   jalr zero, ra, 0
```

(b)

```
.text

main:

   li   a0, 7           # n

   jal  ra, fib

   mv   s2, a0          # y


   li   a0, 1
```

```
    mv   a1, s2
    ecall              # print y

    li   a0, 10
    ecall              # exit

fib:
    addi sp, sp, -12
    sw   ra, 0(sp)
    sw   s0, 4(sp)
    sw   s1, 8(sp)

    li   t0, 2
    blt  a0, t0, fib_end   # n < 2 -> return n

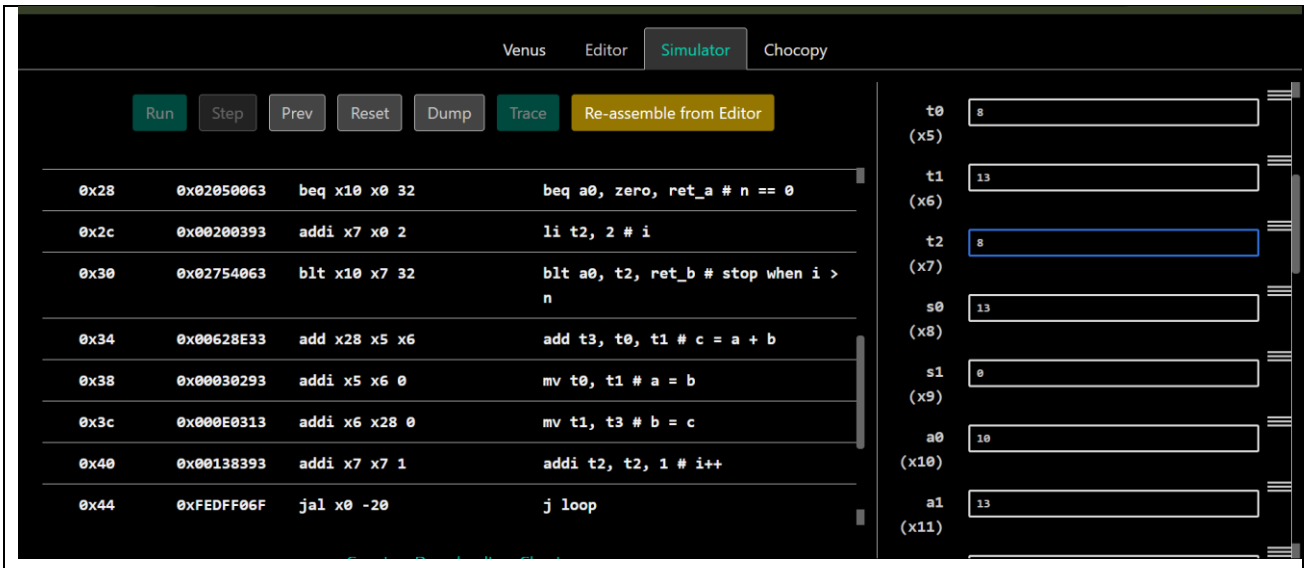
    mv   s0, a0          # keep n
    addi a0, s0, -1
    jal  ra, fib         # fib(n-1)
    mv   s1, a0
```

```
    addi a0, s0, -2
    jal  ra, fib        # fib(n-2)
    add  a0, a0, s1

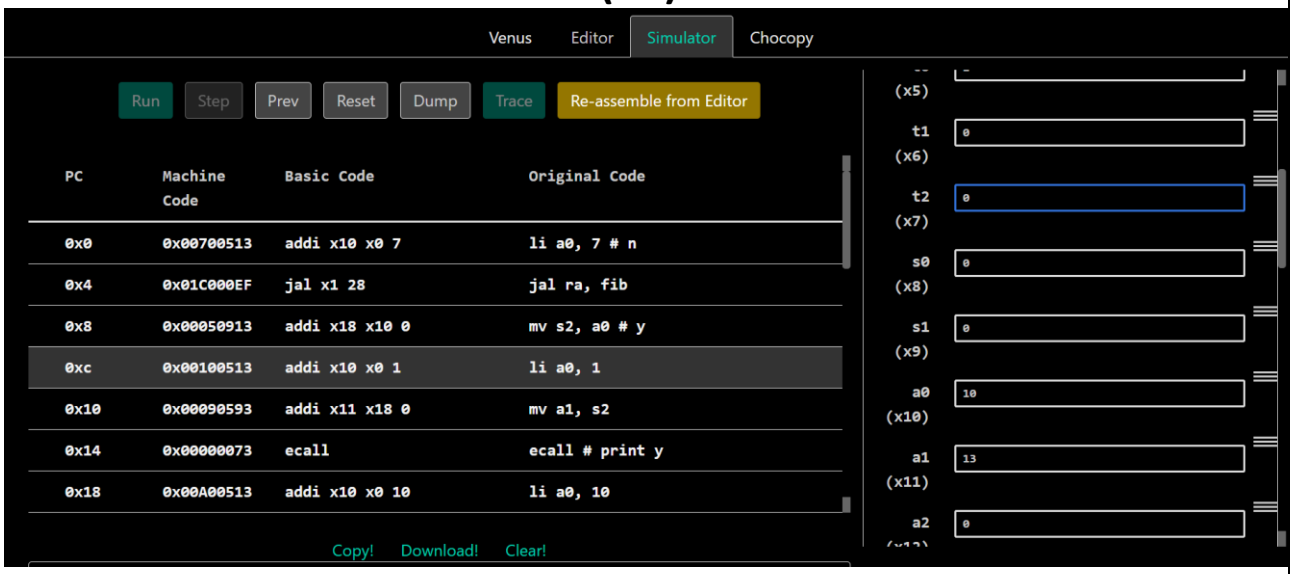

fib_end:
    lw   ra, 0(sp)
    lw   s0, 4(sp)
    lw   s1, 8(sp)
    addi sp, sp, 12
    jalr zero, ra, 0
```

Output:

(8a)


Venus
Editor
Simulator
Chocopy
Run
Step
Prev
Reset
Dump
Trace
Re-assemble from Editor
0x28 0x02050063 beq x10 x0 32 beq a0, zero, ret_a # n == 0
0x2c 0x00200393 addi x7 x0 2 li t2, 2 # i
0x30 0x02754063 blt x10 x7 32 blt a0, t2, ret_b # stop when i > n
0x34 0x00628E33 add x28 x5 x6 add t3, t0, t1 # c = a + b
0x38 0x00030293 addi x5 x6 0 mv t0, t1 # a = b
0x3c 0x000E0313 addi x6 x28 0 mv t1, t3 # b = c
0x40 0x00138393 addi x7 x7 1 addi t2, t2, 1 # i++
0x44 0xFEDFF06F jal x0 -20 j loop
t0 (x5) 8
t1 (x6) 13
t2 (x7) 8
s0 (x8) 13
s1 (x9) 0
a0 (x10) 10
a1 (x11) 13

(8b)


Venus
Editor
Simulator
Chocopy
Run
Step
Prev
Reset
Dump
Trace
Re-assemble from Editor
PC Machine Code Basic Code Original Code
0x0 0x00700513 addi x10 x0 7 li a0, 7 # n
0x4 0x01C000EF jal x1 28 jal ra, fib
0x8 0x00050913 addi x18 x10 0 mv s2, a0 # y
0xc 0x00100513 addi x10 x0 1 li a0, 1
0x10 0x00090593 addi x11 x18 0 mv a1, s2
0x14 0x00000073 ecall ecall # print y
0x18 0x00A00513 addi x10 x0 10 li a0, 10
Copy!
Download!
Clear!
t1 (x6) 0
t2 (x7) 0
s0 (x8) 0
s1 (x9) 0
a0 (x10) 10
a1 (x11) 13
a2 0